

Programación III — Apunte Guía de Pizarrón

Origen de la Notación Asintótica y Ejemplos Concretos de Código

Cátedra: Programación III

Uso: Material Teórico-Práctico / Pizarrón

1. ¿Por qué se denomina "Notación Asintótica"?

El término "**asintótico**" proviene de la matemática (geometría y análisis matemático), donde una **asíntota** es una línea a la cual una función o curva se aproxima indefinidamente a medida que una variable tiende hacia un límite, por lo general hacia el infinito ($n \rightarrow \infty$).

En ciencias de la computación, cuando evaluamos la eficiencia de un algoritmo, no nos interesa el rendimiento exacto para valores pequeños de entrada ($n = 2$ o $n = 5$). Nos interesa estudiar el comportamiento de la función de tiempo $T(n)$ cuando el tamaño de la entrada se vuelve extremadamente grande ($n \rightarrow \infty$).

Los tres pilares del comportamiento asintótico:

- Análisis de Crecimiento en el Límite:** Se estudia el comportamiento frontera de la función cuando n crece sin límite.
- Descarte de Términos de Menor Orden:** Si una función de tiempo es $T(n) = 3n^2 + 50n + 1000$, a medida que n crece, el término $3n^2$ domina abrumadoramente. Por lo tanto, $50n + 1000$ resulta insignificante.
- Independencia de las Constantes Multiplicativas:** Las constantes dependen del hardware o del compilador. El análisis asintótico ignora coeficientes constantes para enfocarse en el *orden de magnitud* puro.

2. Clasificación de las Cotas Asintóticas (O , Ω , Θ)

Notación	Nombre Técnico	Significado Conceptual	Definición Formal
Big-O (O)	Cota Superior Asintótica	Garantía del Peor Caso . El algoritmo no tardará más que esto.	$T(n) \leq c \cdot f(n)$ para todo $n \geq n_0$
Big-Omega (Ω)	Cota Inferior Asintótica	Escenario del Mejor Caso . El algoritmo tardará al menos esto.	$T(n) \geq c \cdot f(n)$ para todo $n \geq n_0$
Big-Theta (Θ)	Cota Ajustada / Estricta	Comportamiento Exacto . Acotado superior e inferiormente por la misma función.	$c_1 \cdot f(n) \leq T(n) \leq c_2 \cdot f(n)$

3. Ejemplos Concretos para Pizarrón (Estilo Java / C++)

Ejemplo 1: Costo Constante $O(1)$ — Acceso Directo por Índice

El tiempo de ejecución no depende del tamaño del arreglo n .

```
public int obtenerPrimerElemento(int[] arreglo) {  
    return arreglo[0]; // Operación elemental única:  $O(1)$   
}
```

Explicación para Pizarrón: Sin importar si el arreglo contiene 10 o 10.000.000 de elementos, acceder a la posición [0] requiere una única operación de cálculo de dirección de memoria.

Ejemplo 2: Costo Lineal $O(n)$ — Recorrido Simple o Búsqueda Lineal

El número de operaciones crece de forma directamente proporcional al tamaño de la entrada n .

```
public int buscarElemento(int[] arreglo, int x) {
    for (int i = 0; i < arreglo.length; i++) { // Se repite hasta n veces
        if (arreglo[i] == x) {
            return i;
        }
    }
    return -1;
}
```

Explicación para Pizarrón:

- **Mejor caso ($\Omega(1)$):** El elemento x está en la primera posición (1 comparación).
- **Peor caso ($O(n)$):** El elemento no está o está al final (requiere n comparaciones).

Ejemplo 3: Costo Logarítmico $O(\log n)$ — Búsqueda Binaria

En cada paso, el espacio de búsqueda se reduce a la mitad ($n/2$).

```
public int busquedaBinaria(int[] arreglo, int x) {
    int inicio = 0, fin = arreglo.length - 1;
    while (inicio <= fin) {
        int medio = inicio + (fin - inicio) / 2;
        if (arreglo[medio] == x) return medio;
        if (arreglo[medio] < x) inicio = medio + 1;
        else fin = medio - 1;
    }
    return -1;
}
```

Explicación para Pizarrón: Si $n = 1000$, en ~ 10 pasos se encuentra el elemento ($2^{10} \approx 1024$). Si $n = 1.000.000$, solo requiere ~ 20 pasos ($\log_2(1.000.000) \approx 20$).

Ejemplo 4: Costo Cuadrático $O(n^2)$ — Bucles Anidados (Bubble Sort / Selección)

Por cada elemento del arreglo, se realiza un recorrido completo sobre el resto de los elementos.

```
public void imprimirPares(int[] arreglo) {
    int n = arreglo.length;
    for (int i = 0; i < n; i++) { // Bucle externo: n veces
        for (int j = 0; j < n; j++) { // Bucle interno: n veces
            System.out.println(arreglo[i] + ", " + arreglo[j]);
        }
    }
}
```

Explicación para Pizarrón: Total de iteraciones: $n \times n = n^2$. Si $n = 100$, realiza 10.000 operaciones. Si $n = 1000$, realiza 1.000.000 de operaciones.